

EX200

Guia de Estudos — Dia 03/05

LVM · VDO · Thin Provisioning · Swap · Partição Padrão

B1

LVM Padrão

B2

PE Size
Custom

B3

Partição Padrão

B4

Swap

B5

VDO +
Thin

HIERARQUIA LVM — MEMORIZE

PV

VG

LV

Physical Volume — tijolo (partição) Volume Group — pilha de tijolos Logical Volume — estrutura construída

ARMADILHAS CRITICAS

- ✓ -L (maiusc) = tamanho em MB/GB
- ✓ lvextend -r = expande LV + FS
- ✓ Swap fstab: 'none' como mountpoint
- ✓ Particao padrao: mkfs direto

- ✗ -l (minusc) = numero de extents
- ✗ sem -r = apenas LV, FS nao cresce
- ✗ Nao usar /swap como mountpoint
- ✗ Nao usar pvcreate em particao padrao

B1

LVM Padrão — Fluxo Completo

`fdisk → pvcreate → vgcreate → lvcreate → mkfs → fstab → lvextend -r`

Hierarquia Mental

PV (tijolo) → VG (pilha de tijolos) → LV (estrutura construída). Esta hierarquia é sempre a mesma em todos os simulados. O que muda são os nomes (`myvg/vgmyvg/vgroup/vg1/vg6`) e os tamanhos.

Fluxo completo com explicações

```
# PASSO 1: Criar partição
fdisk /dev/sdb
  n → nova partição
  p → primary (ou Enter em GPT)
  Enter → número padrão
  Enter → setor inicial padrão
  +2G → tamanho (ajuste conforme questão)
  t → mudar tipo
  8e → Linux LVM (MBR) | 31 em GPT
  w → gravar e sair
partprobe /dev/sdb    # forçar kernel a reler tabela de partições

# PASSO 2: Physical Volume
pvcreate /dev/sdb1

# PASSO 3: Volume Group (PE size padrão 4MB)
vgcreate myvg /dev/sdb1

# PASSO 4: Logical Volume
lvcreate -n mylv -L 500M myvg    # -L maiúsculo = tamanho
lvcreate -n mylv -l 50    myvg    # -l minúsculo = nº extents

# PASSO 5: Formatar
mkfs.ext4 /dev/myvg/mylv
mkfs.xfs /dev/myvg/mylv

# PASSO 6: Montar persistente
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a    # testar fstab sem reboot
df -h /mylv    # confirmar montagem

# PASSO 7: Expandir LV + filesystem em UM comando
lvextend -r -L +500M /dev/myvg/mylv    # adicionar 500MB
lvextend -r -L 1G    /dev/myvg/mylv    # definir tamanho total

# Verificação final
pvs && vgs && lvs
lsblk
```

■ **lvextend sem -r expande apenas o container LV. O filesystem NÃO cresce. SEMPRE use -r.**

■ **partprobe é obrigatório após fdisk — sem ele o kernel não reconhece a nova partição e os próximos passos falham.**

S1 · Q6

Disco /dev/sdb | VG myvg | LV mylv 500MiB → ext4 → /mylv | Expandir +500MiB

```
fdisk /dev/sdb    # n → p → Enter → Enter → +1G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate myvg /dev/sdb1
lvcreate -n mylv -L 500M myvg
mkfs.ext4 /dev/myvg/mylv
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a && df -h /mylv
lvextend -r -L +500M /dev/myvg/mylv
df -h /mylv      # confirmar novo tamanho
```

■ Crie a partição com +1G para ter espaço para 500M inicial + expansão de +500M.

S2 · Q7

Disco /dev/sdb | Partição 4GiB | VG vgmyvg | LV lvmylv 1GiB → ext4 → /lvmylv | Expandir +500MiB

```
fdisk /dev/sdb    # n → Enter → Enter → +4G → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate vgmyvg /dev/sdb1
lvcreate -n lvmylv -L 1G vgmyvg
mkfs.ext4 /dev/vgmyvg/lvmylv
mkdir /lvmylv
echo "/dev/vgmyvg/lvmylv /lvmylv ext4 defaults 0 0" >> /etc/fstab
mount -a
lvextend -r -L +500M /dev/vgmyvg/lvmylv
df -h /lvmylv
```

S3 · Q10

Disco /dev/sdb partição /dev/sdb2 4GiB | VG vgroup | LV lvol 1GiB → ext4 → /lvol | Expandir +100MiB

```
fdisk /dev/sdb    # n → Enter → Enter → +4G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb2
vgcreate vgroup /dev/sdb2
lvcreate -n lvol -L 1G vgroup
mkfs.ext4 /dev/vgroup/lvol
mkdir /lvol
echo "/dev/vgroup/lvol /lvol ext4 defaults 0 0" >> /etc/fstab
mount -a
lvextend -r -L +100M /dev/vgroup/lvol
df -h /lvol
```

S4 · Q10

Disco /dev/sdb | VG vg1 | LV lv1 1GiB → ext4 → /lv1 | Expandir +500MiB

```
fdisk /dev/sdb    # n → Enter → Enter → +2G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate vg1 /dev/sdb1
lvcreate -n lv1 -L 1G vg1
mkfs.ext4 /dev/vg1/lv1
mkdir /lv1
echo "/dev/vg1/lv1 /lv1 ext4 defaults 0 0" >> /etc/fstab
mount -a
lvextend -r -L +500M /dev/vg1/lv1
df -h /lv1
```

S6 · Q10

Disco /dev/sdb | VG vg6 | LV lv6 2GiB → ext4 → /lv6 | Expandir +500MiB



```
fdisk /dev/sdb    # n → Enter → Enter → +4G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate vg6 /dev/sdb1
lvcreate -n lv6 -L 2G vg6
mkfs.ext4 /dev/vg6/lv6
mkdir /lv6
echo "/dev/vg6/lv6 /lv6 ext4 defaults 0 0" >> /etc/fstab
mount -a
lvextend -r -L +500M /dev/vg6/lv6
df -h /lv6
```

B2

PE Size Personalizado

`vgcreate -s 16M` | `lvcreate -l (extents)` vs `-L (tamanho)`

O que é PE (Physical Extent)?

PE é o bloco mínimo de alocação do LVM — como a unidade de cluster de um filesystem. O padrão é 4MB. Quando o exame pede PE de 16M, use `-s 16M` no `vgcreate`. LVs desse VG só podem ser criados em múltiplos de 16MB.

A conta que o exame cobra:

- $50 \text{ extents} \times 16\text{MB} = 800\text{MB}$ (tamanho resultante do LV)
- $800\text{MB} \div 16\text{MB} = 50 \text{ extents}$ (conta inversa — também pode cair)

■ `-l` minúsculo = número de extents | `-L` maiúsculo = tamanho em MB/GB | **NÃO confundir!**

S5 · Q15

VG myvg com PE size 16MiB | LV mylv com 50 extents | xfs → /mnt/mylv

```
fdisk /dev/sdb # n → Enter → Enter → +2G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate -s 16M myvg /dev/sdb1 # PE size personalizado
lvcreate -l 50 -n mylv myvg # -l minúsculo = extents
# 50 extents × 16MB = 800MB total
mkfs.xfs /dev/myvg/mylv
mkdir /mnt/mylv
echo "/dev/myvg/mylv /mnt/mylv xfs defaults 0 0" >> /etc/fstab
mount -a
vgdisplay myvg | grep "PE Size" # deve mostrar 16.00 MiB
lvdisplay /dev/myvg/mylv | grep "Current LE" # deve mostrar 50
```

S6 · Q18

VG myvg com PE size 16MiB usando /dev/sdb2 | LV mylv com 50 extents | xfs → /mnt/mylv

```
fdisk /dev/sdb    # criar partição 2, tipo 8e
partprobe /dev/sdb
pvcreate /dev/sdb2
vgcreate -s 16M myvg /dev/sdb2
lvcreate -l 50 -n mylv myvg
mkfs.xfs /dev/myvg/mylv
mkdir /mnt/mylv
echo "/dev/myvg/mylv /mnt/mylv xfs defaults 0 0" >> /etc/fstab
mount -a
vgdisplay myvg | grep "PE Size"
lvdisplay /dev/myvg/mylv | grep "Current LE"
```

■ Mesmo padrão do S5·Q15 mas usando /dev/sdb2 (segunda partição). Atenção ao número da partição.

B3

Partição Padrão — sem LVM

mkfs direto na partição · UUID no fstab · NÃO usar pvcreate

Quando NÃO usar LVM

Quando a questão pede 'partição padrão', 'formatar diretamente', ou não menciona PV/VG/LV — você formata a partição diretamente, sem criar estrutura LVM. O fluxo é: fdisk → mkfs (pula tudo do meio).

Por que UUID no fstab?

Nomes de dispositivo como /dev/sdb1 podem mudar entre reboots dependendo da ordem de detecção dos discos. O UUID é permanente — identifica a partição pelo conteúdo, não pelo nome do dispositivo. Use blkid para obter o UUID.

S3 · Q6

Criar partição 512MiB em /dev/sdb → formatar ext4 → montar em /mnt/data usando UUID → persistente

```
fdisk /dev/sdb # n → Enter → Enter → +512M → w
partprobe /dev/sdb
mkfs.ext4 /dev/sdb1 # NÃO usar pvcreate aqui!
mkdir -p /mnt/data
blkid /dev/sdb1 # copiar o UUID mostrado
# lsblk -f /dev/sdb1 # alternativa ao blkid
vim /etc/fstab
# Adicionar linha:
# UUID=<uuid-aqui> /mnt/data ext4 defaults 0 0
mount -a
df -h /mnt/data
```

■ Esta questão pede partição PADRÃO. NÃO executar pvcreate/vgcreate/lvcreate.

S6 · Q21

Criar partição 2GiB em /dev/sdb → formatar ext4 → montar em /mnt/drive usando UUID → persistente



```
fdisk /dev/sdb    # n → Enter → Enter → +2G → w
partprobe /dev/sdb
mkfs.ext4 /dev/sdb1    # partição direta, sem LVM
mkdir -p /mnt/drive
blkid /dev/sdb1        # copiar UUID
vim /etc/fstab
# UUID=<uuid> /mnt/drive ext4 defaults 0 0
mount -a
df -h /mnt/drive
```

B4

Swap — Todos os Simulados

fdisk tipo 82 → mkswap → UUID no fstab → swapon -a

Três diferenças em relação a partições comuns:

- **Tipo no fdisk** — 82 (Linux Swap), não 8e (Linux LVM)
- **Formatador** — mkswap, não mkfs
- **fstab** — ponto de montagem = none | tipo = swap

Fluxo padrão Swap (igual em todos os simulados):

```
fdisk /dev/sdb
n → nova partição
Enter → número padrão
Enter → setor inicial padrão
+512M → tamanho (512M / 500M / 1G conforme questão)
t → mudar tipo
82 → Linux Swap
w → gravar
partprobe /dev/sdb
mkswap /dev/sdb2      # ajustar número da partição
blkid /dev/sdb2       # copiar UUID
vim /etc/fstab
# UUID=<uuid> none swap defaults 0 0
#          ^^^^^
#          SEMPRE 'none' — nunca /swap ou outro caminho
swapon -a             # ativar sem reboot
free -h               # confirmar coluna Swap aumentou
```

■ O mount point para swap é 'none' — não /swap nem /swapfile. O tipo de fs é 'swap'. Erro aqui impede ativação no boot.

■ swapon -a ativa tudo do fstab com tipo swap — garante que o fstab está correto. swapon /dev/sdb2 ativa só o dispositivo

Simulado	Questão	Tamanho	Partição típica
S1	Q15	512MiB	/dev/sdb2
S2	Q20	500MiB	/dev/sdb2
S5	Q18	1GiB	/dev/sdb2
S6	Q22	1GiB	/dev/sdb2

■ O número da partição (/dev/sdb2) depende de quantas partições já existem no disco. Se o disco tem /dev/sdb1 (LVM), a

B5

VDO & Thin Provisioning

Deduplicação/compressão (VDO) · Over-provisioning (Thin) · lvrename

VDO — Virtual Data Optimizer

VDO é uma tecnologia de deduplicação e compressão inline gerenciada pelo LVM no RHEL 10. Dois parâmetros críticos e independentes:

- **--size** = espaço físico real consumido no VG (o que você tem de disco)
- **--virtualsize** = espaço apresentado ao SO (pode ser muito maior — fator 4x a 10x)

■ **RHEL 10 exige mínimo ~5GB físico para VDO. O índice UDS de deduplicação consome vários GB de metadados. Não te**

S1 · Q20

VG myvg já existente | VDO LV vdo_lv | 5GiB físico → 20GiB virtual | xfs → /vdo_data

```
dnf install lvm2 vdo -y
# VG já existe (myvg) — apenas criar o LV VDO
lvcreate --type vdo --name vdo_lv --size 5G --virtualsize 20G myvg
mkfs.xfs /dev/myvg/vdo_lv
mkdir /vdo_data
echo "/dev/myvg/vdo_lv /vdo_data xfs defaults 0 0" >> /etc/fstab
mount -a
vdostats --human-readable          # ver taxa compressão/deduplicação
lsblk
```

S4 · Q11

Disco /dev/sdc | VG vdovg | VDO LV myvdo | 5GiB físico → 50GiB virtual | xfs → /mydir

```
dnf install lvm2 vdo -y
pvcreate /dev/sdc
vgcreate vdovg /dev/sdc
lvcreate --type vdo --name myvdo --size 5G --virtualsize 50G vdovg
mkfs.xfs /dev/vdovg/myvdo
mkdir /mydir
echo "/dev/vdovg/myvdo /mydir xfs defaults 0 0" >> /etc/fstab
mount -a
vdostats --human-readable
lvs -o+vdo_compression,vdo_deduplication
```

Thin Provisioning

Thin provisioning permite prometer mais espaço do que existe fisicamente. Dois objetos LVM: Thin Pool (container físico, criado com -T e -L) e Thin Volume (virtual, criado com -V e -T referenciando o pool). Thick reserva espaço imediatamente. Thin só ocupa conforme dados são escritos.

S3 · Q27 — Thin básico

VG thin_vg em /dev/sdb3 | Thin Pool my_thin_pool 500MiB | Thin Volume thin_vol 2GiB virtual | xfs → /mnt/thin

```
fdisk /dev/sdb # criar partição 3, +1G, tipo 8e
partprobe /dev/sdb
pvcreate /dev/sdb3
vgcreate thin_vg /dev/sdb3

# Criar thin pool (500MB físico real)
lvcreate -L 500M -T thin_vg/my_thin_pool

# Criar thin volume DENTRO do pool (2GB virtual – maior que o pool!)
lvcreate -V 2G -T thin_vg/my_thin_pool -n thin_vol

mkfs.xfs /dev/thin_vg/thin_vol
mkdir /mnt/thin
echo "/dev/thin_vg/thin_vol /mnt/thin xfs defaults 0 0" >> /etc/fstab
mount -a
lvs thin_vg # mostra VSize (2G) e pool (500M)
```

■ -T cria/referencia thin pool. -V define o tamanho virtual. Sem -T na criação do pool: não é thin. Sem -V no volume: não é

S6 · Q11 — Thin avançado com lvrename

VG vg1 em /dev/sdb2 | Thin Pool thinpool1 2GiB | Thin Volume thinvol1 1TiB virtual | Expandir pool +500MiB | Renomear pool→thinpool2 e volume→thinvol2 | xfs → /thinvol2

```
fdisk /dev/sdb # criar partição 2, tipo 8e, +4G
partprobe /dev/sdb
pvcreate /dev/sdb2
vgcreate vg1 /dev/sdb2

# Criar pool e volume thin
lvcreate -L 2G --thinpool vg1/thinpool1
lvcreate -V 1T -T vg1/thinpool1 -n thinvol1

# Expandir o pool +500MB
lvextend -L +500M vg1/thinpool1

# Renomear pool e volume (lvrename VG/antigo VG/novo)
lvrename vg1/thinpool1 vg1/thinpool2
lvrename vg1/thinvol1 vg1/thinvol2

mkfs.xfs /dev/vg1/thinvol2
mkdir /thinvol2
echo "/dev/vg1/thinvol2 /thinvol2 xfs defaults 0 0" >> /etc/fstab
mount -a
lvs vg1 # confirmar nomes e tamanhos
```

■ **lvrename funciona em pools e volumes thin. Sintaxe: `lvrename VG/LV_ANTIGO VG/LV_NOVO` — VG explícito obrigatório**

■ **NÃO use `mv` no `/dev` — isso só renomeia o symlink, não os metadados LVM internos.**

Tabela de Diferenças Críticas

Situação	Comando CORRETO	Erro comum
LV por tamanho	<code>lvcreate -L 500M myvg</code>	<code>lvcreate -l 500M (errado)</code>
LV por extents	<code>lvcreate -l 50 myvg</code>	<code>lvcreate -L 50 (errado)</code>
Expandir + FS	<code>lvextend -r -L +500M ...</code>	Esquecer o <code>-r</code>
PE personalizado	<code>vgcreate -s 16M vg /dev/sdb1</code>	Criar VG sem <code>-s</code>
Swap no fstab	<code>UUID=xx none swap defaults 0 0</code>	<code>UUID=xx /swap swap ...</code>
Partição padrão	<code>mkfs.ext4 /dev/sdb1</code>	<code>pvcreeate /dev/sdb1</code> (cria LVM)
VDO mínimo	<code>--size 5G</code>	<code>--size 1G</code> (falha por metadados)
Thin pool	<code>lvcreate -L 500M -T vg/pool</code>	Esquecer o <code>-T</code>
Thin volume	<code>lvcreate -V 2G -T vg/pool -n vol</code>	Esquecer o <code>-V</code>

■ Checklist — O que dominar ao final do Dia 03/05

■ LVM Padrão

- Fluxo LVM completo (`fdisk` → `pvs` → `vgs` → `lvs`) sem consulta
- `lvextend -r` funciona — filesystem cresce junto com o LV
- fstab correto para LVM (`/dev/vg/lv` formato) + `mount -a` sem erro

■ PE Size Personalizado

- `vgcreate -s 16M` para PE personalizado
- `lvcreate -l 50` (extents) vs `-L 500M` (tamanho) — não confundir
- Saber fazer a conta: $50 \text{ extents} \times 16\text{MB} = 800\text{MB}$

■ Partição Padrão

- `mkfs` direto na partição SEM `pvcreeate/vgcreate/lvcreate`
- UUID no fstab — obter com `blkid /dev/sdb1`

■ Swap

- `fdisk` tipo 82 (swap, não 8e de LVM)
- fstab com 'none' como mountpoint e 'swap' como tipo
- `swapon -a` ativa swap do fstab | `free -h` confirma

■ VDO & Thin Provisioning

- VDO: `--size` (físico) vs `--virtualsize` (apresentado ao SO)
- VDO mínimo 5GB físico (metadados do índice UDS)
- Thin pool: `lvcreate -L -T` | Thin volume: `lvcreate -V -T`

- lvrename vg1/old vg1/new funciona em pools e volumes

RHCSA EX200 · Dia 03/05: LVM + VDO + Thin + Swap + Partição · Prova: 14/05 · Boa sorte!